

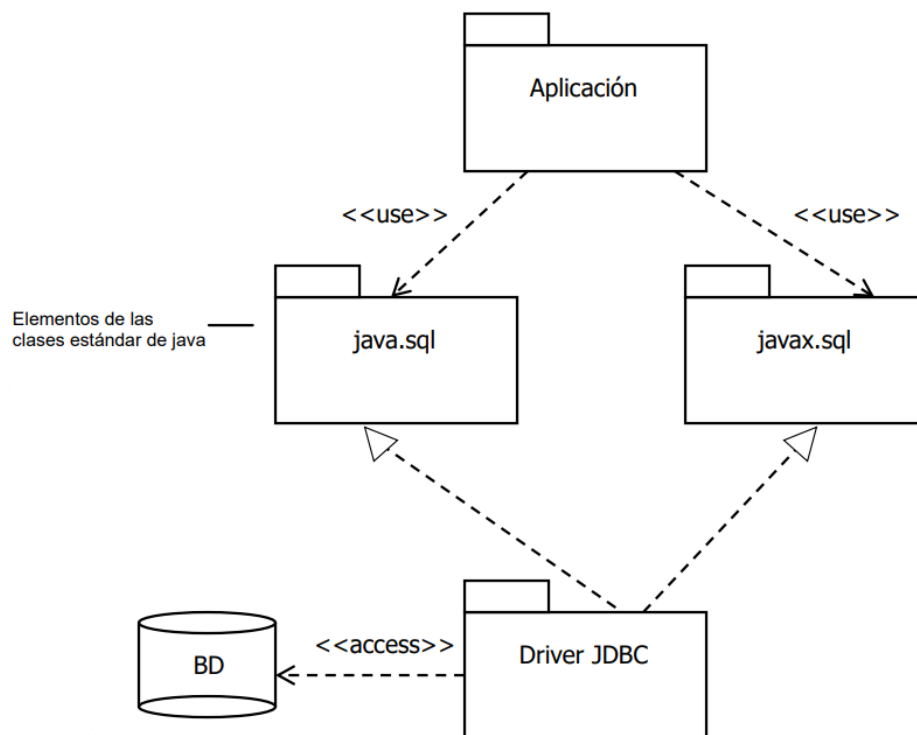
TEMA 2: JDBC

INTRODUCCIÓN

- JDBC → Java DataBase Connectivity → API estándar que permite lanzar consultas a un BD relacional
- Desarrollador → trabaja contra los paquetes **java.sql** y **javax.sql**
 - Forman parte de Java SE
 - Contienen numerosas interfaces y algunas clases concretas que conforman la API de JDBC
- Driver adecuado → conectarse a una BD y lanzar consultas
 - Suele ser un fichero **.jar** que contiene una implementación de todas las interfaces de la API de JDBC
 - Lo proporciona el fabricante de la BD o un tercero

ACCESOS BÁSICOS

DRIVER JDBC



INDEPENDENCIAS DE LA BD

- Si la aplicación cambia de BD hace falta otro drive, no cambiar el código (idealmente)
- Las BBDD relacionales usan diferentes dialectos SQL
 - Tipos de datos → varían según la BD
 - Generación de identificadores → secuencias, autonumerados, etc.
 - Se pueden usar patrones (técnicas) para conseguir que el código sea independiente de la BD

EJECUCIÓN DE SENTENCIAS

- Interfaz **Connection**
 - Representa una conexión en la BD
 - **prepareStatement** → permite construir objetos **PreparedStatement** para lanzar consultas
- Interfaz **PreparedStatement**
 - Contiene la consulta SQL **parametrizada** que se va a lanzar
 - Parámetros → caracteres “?” de la consulta
 - Dispone de métodos **setXXX** para dar valor a los parámetros
 - Los parámetros se numeran de 1 en adelante
 - **executeUpdate**
 - Permite lanzar consultas SQL de actualización (ej. **INSERT**, **UPDATE**, **DELETE**, etc.)
 - Devuelve el nº de filas afectadas por la actualización
 - **executeQuery**
 - Permite lanzar consultas SQL de lectura (ej. **SELECT**)
- Formateo de datos con **PreparedStatement**
 - Se lanza la consulta → el driver sustituye y **formatea automáticamente** los parámetros en el formato requerido por la BD
 - Cómodo → el driver formatea los datos (no el desarrollador)
 - Portable → el driver usará el formato adecuado para la BD subyacente
 - Ejemplo 1 → cadenas de caracteres y n^{os} enteros
 - En **InsertExample**, la consulta que se lanza en la primera iteración del bucle es


```
INSERT INTO TutMovie (movieId, title, runtime)
VALUES ('movie-1', 'movie-1 title', 90)
```
 - El driver entrecomilla los 2 primeros valores (son de tipo cadena de caracteres) y no aplica ningún formato especial al tercer valor (es de tipo entero)
 - Ejemplo 2 → fechas
 - Se supone:
 - La tabla **User** incluye la columna **birthdate** de tipo **DATE**
 - La variable **birthdate** contiene la fecha “1 de Septiembre de 2015”
 - La base de datos es MySQL
 - Si se ejecuta


```
...
LocalDate birthdate = ...
String queryString = "INSERT INTO User (birthdate, ...) " +
    "VALUES (?, ...)";

PreparedStatement preparedStatement =
    connection.prepareStatement(queryString);
preparedStatement.setDate(1, Date.valueOf(birthdate));
preparedStatement.executeUpdate();
...
```
 - El driver lanza


```
INSERT INTO User (birthdate, ...) VALUES ('2015-09-01', ...)
```

SQLException

- Los métodos de la API de JDBC reportan cualquier error lanzando **SQLException** (“checked”) o una de sus hijas

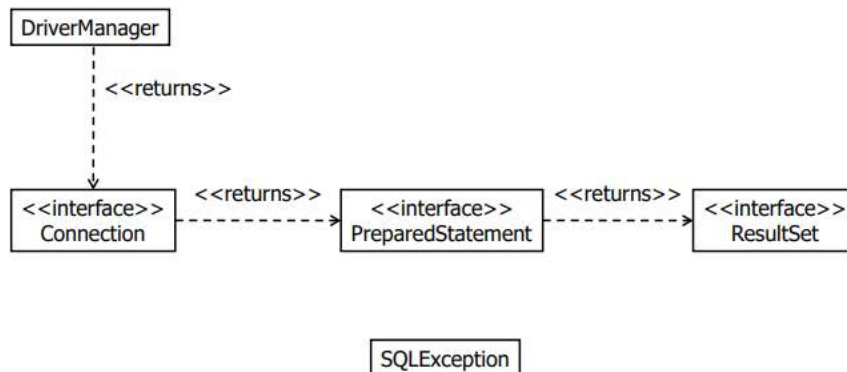
PROCESAMIENTO DE FILAS RESULTADO

- Interfaz **ResultSet** → representa todas las filas que han concordado con la consulta de búsqueda
- Iteración sobre filas
 - Implementación **ResultSet** → mantiene un cursor, inicialmente posicionado antes de la primera fila
 - Si no quedan filas por leer → **next** devuelve **false**
 - Si quedan filas por leer → avanza el cursor en una posición y devuelve **true**
- Dispone de métodos **getXXX** para acceder a los valores de las columnas de la fila en la que está posicionado el cursor

OBTENCIÓN DE CONEXIONES

- La clase **ConnectionManager** proporcionada en el ejemplo facilita la obtención de conexiones con la API básica de JDBC
- **java.sql.DriverManager** → dispone del método estático **getConnection** que permite obtener una conexión a la BD a partir de
 - URL (formato especificado en la documentación del driver) → indica la máquina en la que corre la BD y el nombre del esquema
 - Identificador y contraseña de un usuario que tenga permisos de acceso
- Aplicación real → URL, usuario y contraseña se leen de un fichero de configuración

RESUMEN PRINCIPALES ABSTRACCIONES DE LA API DE JDBC



LIBERACIÓN DE RECURSOS

- **try-with-resources** → construcción utilizada por los ejemplos para abrir/cerrar conexiones
- **try-with-resources**
 - Extensión de la construcción **try-catch** introducida en Java 7
 - Permite declarar 1 o + “recursos” después de la palabra clave **try** → los recursos se declaran entre paréntesis separados por “;” (___; ___)
 - “Recursos” → objetos que implementan la interfaz **java.lang.AutoCloseable** → solo dispone del método **close**
 - Los recursos deben crearse cuando se declaran
 - El compilador de Java garantiza que se invocará al método **close** de los recursos declarados cuando se termine el bloque **try** (si se producen o no excepciones)
 - Cuando se ejecuten los posibles bloques **catch/finally**, los recursos declarados ya estarán cerrados
- Los ejemplos declaran el recurso “conexión JDBC en la construcción **try-with-resources**
 - **java.sql.Connection** → extiende **java.lang.AutoCloseable**
 - Termina la ejecución del bloque **try** → el código generado por el compilador invoca al método **close** de la conexión
- En consecuencia, se cierra “todo”
- Si no se utiliza **try-with-resources**, habría que cerrar la conexión explícitamente

```

Connection connection = null;

try {
    connection = DriverManager.getConnection();
    << Interactuar con la BD >>
} catch (Exception e) {
    e.printStackTrace(System.err);
} finally {
    try {
        if (connection != null) {
            connection.close();
        }
    } catch (Exception e) {
        e.printStackTrace(System.err);
    }
}
  
```

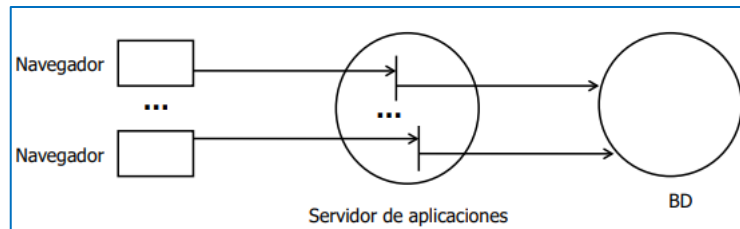
- **finalize**

- Implementación de la interfaz **Connection** → debe redefinir **finalize** para invocar al método **close** en caso de que el desarrollador no lo haya hecho
 - **finalize** es un método definido en **Object** → el recolector de basura lo invoca antes de eliminar un objeto de memoria
- Se crea una **ilusión** de que se podría interactuar con la BD sin cerrar la conexión explícita ni implícitamente vía **try-with-resources**

```
try {
    Connection connection = DriverManager.getConnection();
    << Interactuar con la BD >>
} catch (Exception e) {
    e.printStackTrace(System.err);
}
```

- En un caso real no es buena idea

- Suponemos una aplicación servidora multi-thread, donde cada thread puede tener que acceder a la BD
- Ej. servicio/aplicación Web Java
 - Los servicios/aplicaciones Web Java se ejecutan normalmente dentro de servidores de aplicaciones
 - Un servidor de aplicaciones atiende cada petición HTTP en un thread
 - Es posible atender múltiples peticiones concurrentemente



- Un gestor de BD no puede tener abiertas + de un determinado nº "n" de conexiones
- Si cada thread accede a la BD no cierra la conexión una vez termine su trabajo → la conexión no se cerrará hasta que el recolector de basura elimine esa conexión (se ha quedado sin referenciar)
- Puede ocurrir que se hayan procesado "n" peticiones HTTP y que sus respectivas conexiones todavía no hayan sido eliminadas por el recolector de basura
 - El recolector de basura decide eliminar memoria cuando lo considera oportuno (ej. cuando se lleva consumida cierta cantidad de memoria)
- Cuando llegue la siguiente petición, **DriverManager.getConnection** devolverá **SQLException** porque la BD no admite + conexiones
 - Las "n" conexiones anteriores todavía no se han liberado, a pesar de que no se están usando
- Conclusión → cada thread debe liberar la conexión inmediatamente una vez termine de interactuar con la BD, explícitamente o mediante **try-with-resources**

TIPOS SQL Y JAVA

- **ResultSet** y **PreparedStatement** proporcionan métodos **getXXX** y **setXXX**
- Correspondencia entre tipos Java y SQL estándar
 - Un dato de tipo Java se puede almacenar en una columna cuyo tipo SQL sea consistente con el tipo Java



Tipo Java	Tipo SQL
boolean	BIT
byte	TINYINT
short	SMALLINT
int	INTEGER
long	BIGINT
float	REAL
double	DOUBLE
java.math.BigDecimal	NUMERIC
String	VARCHAR o LONGVARCHAR
byte[]	VARBINARY o LONGVARBINARY
java.sql.Date	DATE
java.sql.Time	TIME
java.sql.Timestamp	TIMESTAMP

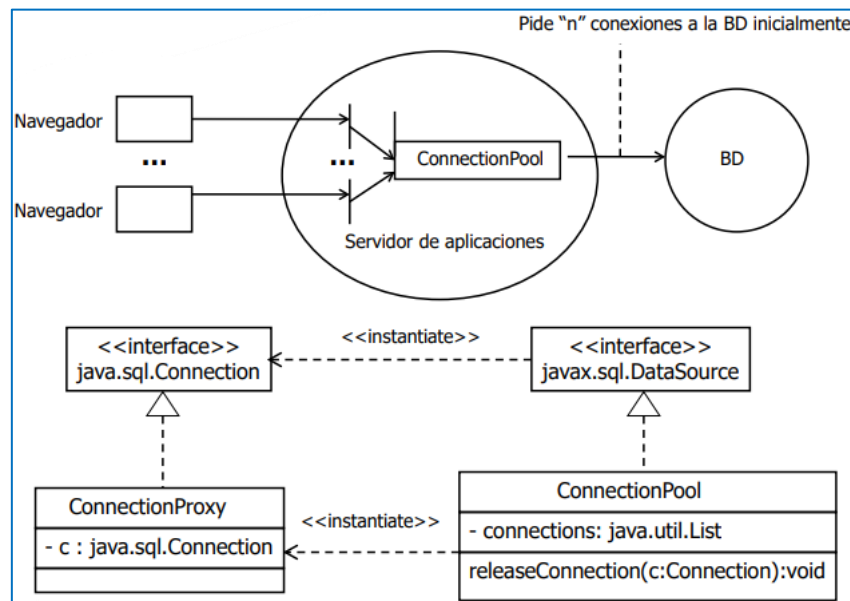
DATASOURCES

- Interfaz **javax.sql.DataSource**
 - Dispone del método **getConnection**
 - Uso de esta interfaz → el desarrollador no tiene que especificar la URL, usuario y contraseña para pedir la conexión
- Los servidores de aplicaciones Java y algunos frameworks ofrecen implementaciones de la interfaz **DataSource**
 - Se utiliza **DriverManager.getConnection** para obtener las conexiones (estrategia compleja)
 - Se utilizan ficheros de configuración para especificar (como mínimo) URL, usuario y contraseña
 - En Jetty o Tomcat (servidores de aplicaciones) se configurarán objetos **DataSource** para acceder a la BD

```
DataSource dataSource = ...
Connection connection = dataSource.getConnection();
```

POOL DE CONEXIONES

- Problema
 - Servidor de aplicaciones que recibe muchas peticiones HTTP por minuto
 - Se puede pedir una conexión a la BD con **DriverManager.getConnection** o el método **getConnection** de un objeto **DataSource**
 - **DriverManager.getConnection** pide una conexión directamente a la BD
 - Operación lenta → se convierte en cuello de botella
 - Implementación básica de **DataSource** → el método **getConnection** invoca **DriverManager.getConnection**
 - Si la BD no admite + conexiones (super el máximo permitido), los métodos **getConnection** devuelven una excepción
- Solución → pool de conexiones
 - Servidores de aplicaciones Java → proporcionan implementaciones de **DataSource** que utilizan la estrategia **pool de conexiones**
 - Objeto **DataSource** → gestiona un conjunto de conexiones que previamente ha solicitado a la BD
 - Estrategia transparente al desarrollador → sólo trabaja contra la interfaz **DataSource**



- **ConnectionPool**
 - Al crearse, pide “n” conexiones a la BD (usando **DriverManager.getConnection**) y las almacena en una lista
 - **getConnection**
 - Quedan conexiones libres en la lista → elige una → la marca como **usada** → devuelve un objeto **ConnectionProxy**
 - No quedan conexiones libres → deja durmiendo (**wait**) al thread llamador
 - **releaseConnection**
 - Devuelve la conexión a la lista → la marca como **libre** → notifica (**notifyAll**) a los posibles threads que esperan por una conexión

- **ConnectionProxy**
 - Proxy de la conexión real
 - **close** → usa **releaseConnection** para devolver la conexión real al pool
 - **finalize** → llama a **ConnectionProxy.close** si no se ha llamado
 - Resto operaciones → delegan en la conexión real
- Observaciones
 - El desarrollado invoca **getConnection** sobre el objeto **DataSource**
 - Hay una conexión libre → se le devuelve rápidamente de la lista (no accede a BD)
 - No hay una conexión libre → el thread llamador se queda dormido hasta que haya una
 - Las conexiones reales no se cierran, se devuelven al pool
- Caídas de la BD
 - La BD cae → las conexiones del pool se invalidan aunque se vuelva a rearrancar la BD (los sockets subyacentes ya no son válidos)
 - Solucionar → la implementación de **getConnection** puede comprobar si la conexión que devuelve **está viva** (es correcta)
 - Opción 1 → haciendo uso de una API específica del fabricante del driver
 - Opción 2 → lanzando una consulta poco costosa a la BD → si no se produce una **SQLException** la conexión es correcta
- Configuración del pool
 - Además de la configuración básica de un **DataSource**, se puede especificar
 - Nº de conexiones a la BD que se solicitan inicialmente
 - Consulta de comprobación de la conexión viva (si se requiere)
 - etc.

TRANSACCIONES

- Permite ejecutar bloques de código con las propiedades ACID (Atomicity-Consistency-Isolation-Durability)
- Por defecto → cuando se crea una conexión está en modo auto-commit
 - Cada consulta lanzada se ejecuta en su propia transacción
- Preciso para ejecutar consultas en una misma transacción
 - Deshabilitar el modo auto-commit de la conexión
 - Lanzar las consultas
 - Terminar con
 - Todo va bien → **connection.commit()**
 - Otro caso → **connection.rollback**

TRANSACTION ISOLATION LEVELS

- **java.sql.Connection** → proporciona el método **setTransactionIsolation** → permite especificar el nivel de aislamiento deseado
 - **TRANSACTION_NONE** → transacciones no soportadas
 - **TRANSACTION_READ_UNCOMMITTED** → pueden ocurrir “dirty reads”, “non-repeatable reads”, “phantom reads”
 - **TRANSACTION_READ_COMMITTED** → pueden ocurrir “non-repeatable reads” y “phantom reads”
 - **TRANSACTION_REPEATABLE_READ** → pueden ocurrir “phantom reads”
 - **TRANSACTION_SERIALIZABLE** → elimina todos los problemas de concurrencia
- Mayor nivel de aislamiento → la BD realiza + bloqueos → menos concurrencia
- Se realizarán transacciones con el nivel de aislamiento **TRANSACTION_SERIALIZABLE** (+ sencillez)
- Existen técnicas que permiten trabajar transaccionalmente en muchas situaciones con un nivel de aislamiento inferior
 - Menos bloqueos en la BD